Proyecto "Infraestructura Viva" – ACME Solutions

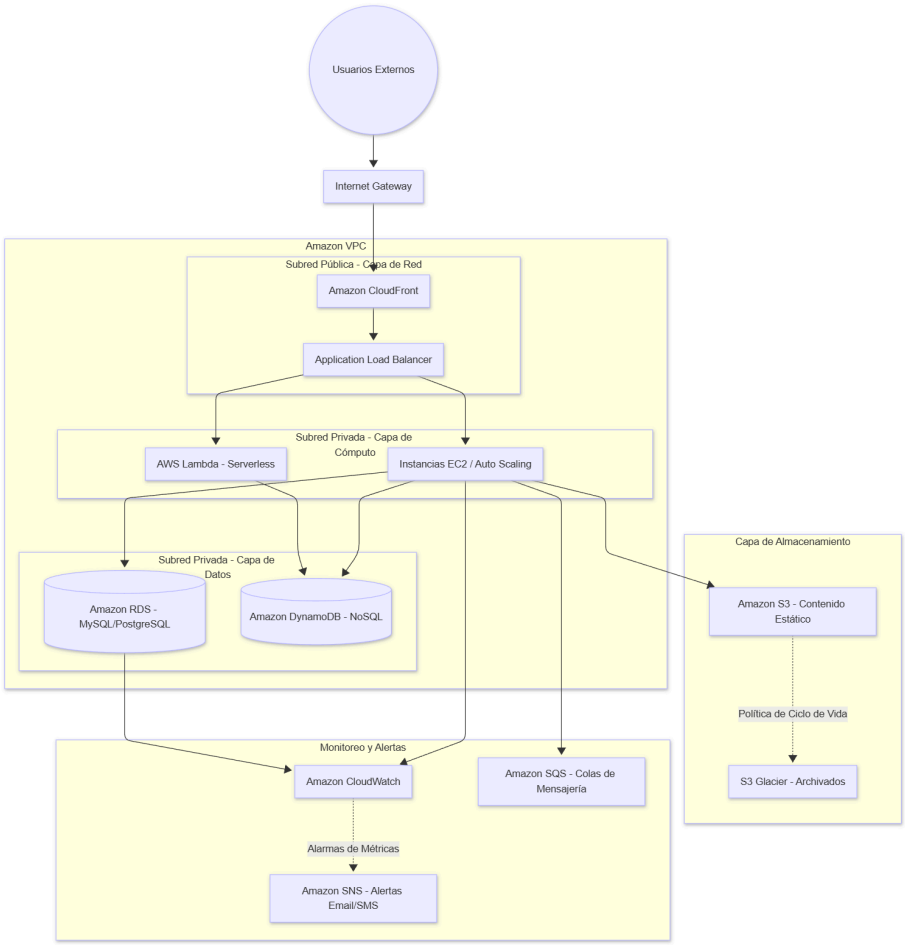
Preparado por: Jimmy Agüero – Estudiante Arquitecto Cloud

Caso: Migración y Modernización de Soluciones Digitales ACME

1. Resumen de la Solución

El objetivo es migrar el entorno *on-premise* actual de los departamentos de ventas, soporte y finanzas a una arquitectura *cloud* basada en AWS. Esta solución busca resolver los problemas de altos costos de mantenimiento y la falta de escalabilidad del hardware físico.

Diagrama


Usuarios Externos
Internet Gateway
Amazon VPC
Subred Pública - Capa de Red
Amazon CloudFront
Application Load Balancer
Subred Privada - Capa de Cómputo
AWS Lambda - Serverless
Instancias EC2 / Auto Scaling
Subred Privada - Capa de Datos
Amazon RDS - MySQL/PostgreSQL
Amazon DynamoDB - NoSQL
Capa de Almacenamiento
Amazon S3 - Contenido Estático
Política de Ciclo de Vida
S3 Glacier - Archivados
Monitoreo y Alertas
Amazon CloudWatch
Amazon SQS - Colas de Mensajería
Alarmas de Métricas
Amazon SNS - Alertas Email/SMS

2. Diseño de Red y Seguridad (VPC)

Para aislar los servicios internos y exponer las aplicaciones de forma segura, se diseñará la siguiente red:

- **Segmentación:** Una VPC con subredes públicas para el balanceo de carga y subredes privadas para proteger los servidores de cómputo y bases de datos.
- **Control de Tráfico:** Implementación de **Security Groups** (firewalls a nivel de instancia) y NACLs (listas de control de acceso a nivel de subred).
- **Conectividad:** Uso de un **Application Load Balancer** para distribuir el tráfico web de manera eficiente.

3. Estrategia de Cómputo y Escalabilidad

Para modernizar el procesamiento de datos sin depender de servidores físicos:

- **Servidores:** Despliegue de aplicaciones en instancias **Amazon EC2** o contenedores, según el requerimiento de la carga de trabajo.
- **Serverless:** Uso de funciones **AWS Lambda** para tareas puntuales, optimizando costos mediante el uso del *Free Tier*.

4. Gestión de Datos (Relacional y NoSQL)

La arquitectura integra dos tipos de persistencia para cubrir todas las necesidades del negocio:

- **Amazon RDS:** Motor relacional (MySQL o PostgreSQL) para transacciones financieras y de ventas, configurado con **backups automáticos** y alta disponibilidad.
- **Amazon DynamoDB:** Base de datos NoSQL para manejar datos semiestructurados con alta velocidad de respuesta mediante índices secundarios.
- **Validación:** Las consultas SQL serán previamente testeadas en la herramienta **SQLiteOnline** para asegurar su compatibilidad.

5. Almacenamiento y Ciclo de Vida

Diferenciación de datos según su frecuencia de uso para optimizar el presupuesto:

- **Amazon S3:** Almacenamiento de objetos para archivos estáticos y multimedia.
- **Políticas de Ciclo de Vida:** Automatización para mover datos antiguos o respaldos históricos hacia **S3 Glacier**, reduciendo drásticamente los costos de almacenamiento a largo plazo.

6. Monitoreo, Mensajería y Alertas

Para una respuesta proactiva ante incidentes:

- **CloudWatch:** Recopilación de métricas críticas como uso de CPU y errores en logs.
- **Amazon SNS/SQS:** Configuración de un canal de notificaciones (email/SMS) y colas de mensajería para desacoplar servicios y garantizar la entrega de mensajes.

Prototipo y Demo Funcional

Este componente técnico demuestra la capacidad de integrar servicios de cómputo con persistencia de datos y herramientas de desarrollo local.

1. Instrucciones de Despliegue: Cómputo y Base de Datos

Para cumplir con el requerimiento de conectar una instancia a una base de datos gestionada , seguiremos estos pasos utilizando el **AWS Free Tier** y **AWS Academy**:

- **Paso 1: Configuración de la Red (VPC):** Crear una VPC con una subred pública y una privada para asegurar que la base de datos no sea accesible directamente desde Internet.
- **Paso 2: Despliegue de Amazon RDS:** Lanzar una instancia de base de datos relacional (ej. MySQL o PostgreSQL) en la subred privada.
- **Paso 3: Despliegue de Cómputo (EC2 o Lambda):** * **Opción VM (EC2):** Lanzar una instancia Linux t2.micro en la subred pública para actuar como servidor de aplicaciones.
    - **Opción Serverless (Lambda):** Configurar una función Lambda dentro de la misma VPC para acceder a la base de datos.
- **Paso 4: Grupos de Seguridad:** Configurar una regla de entrada en el **Security Group** de RDS que permita el tráfico en el puerto 3306 (MySQL) o 5432 (PostgreSQL) únicamente desde el Security Group de la instancia EC2 o la función Lambda.

---

2. Configuración en Visual Studio Code

Se utiliza VS Code para la automatización y gestión de scripts. Un ejemplo de configuración para conectar la aplicación con la base de datos es el uso de archivos de variables de entorno:

Ejemplo de archivo `.env` en VS Code:

```
# Configuración de conexión para la instancia de cómputo
DB_HOST=infra-viva-db.cluster-xxxx.us-east-1.rds.amazonaws.com
DB_PORT=3306
DB_USER=admin_acme
DB_PASSWORD=password_seguro_123
DB_NAME=soluciones_digitales_acme
```

Scripts de Automatización: En VS Code se pueden desarrollar scripts en Python (usando `boto3`) para automatizar el encendido/apagado de instancias y optimizar costos

3. Demostración de Consultas en SQLiteOnline

Para validar el diseño del modelo relacional antes del despliegue en RDS, se utiliza **SQLiteOnline**.

Script de prueba para validación:

```sql
-- 1. Creación de tabla para el departamento de ventas
CREATE TABLE ventas_acme (
    id_venta INTEGER PRIMARY KEY,
    producto TEXT NOT NULL,
    monto DECIMAL(10,2),
    fecha_venta DATE
);

-- 2. Inserción de datos de prueba
INSERT INTO ventas_acme (producto, monto, fecha_venta)
VALUES ('Licencia Cloud', 1500.00, '2026-03-09');

-- 3. Consulta de validación requerida
SELECT producto, SUM(monto) as total_ventas
FROM ventas_acme
GROUP BY producto;
```

Esta herramienta permite certificar que la lógica de las consultas es correcta para los requerimientos de los departamentos de finanzas y ventas.

Resumen de Validación Técnica

- **Escalabilidad:** Se realizarán pruebas de ejecución bajo carga para asegurar que el entorno de cómputo responda a los requerimientos crecientes.
- **Costo:** Toda la demo se mantiene dentro de los límites del **AWS Free Tier** y recursos de **AWS Academy** para garantizar la viabilidad del proyecto.

3. Plan de Monitoreo y Notificaciones

3.1. Estrategia de Observabilidad

El sistema de monitoreo se basa en **Amazon CloudWatch**, centralizando métricas de rendimiento y salud de los recursos para garantizar la continuidad operativa.

3.2. Configuración de Métricas Clave

Se han seleccionado los siguientes umbrales críticos para monitorear la salud del sistema:

Recurso	Métrica	Umbral Crítico	Acción
Cómputo (EC2)	CPUUtilization	> 80% por 5 min	Notificar vía SNS (Alerta de saturación).
Base de Datos (RDS)	FreeStorageSpace	< 5 GB disponibles	Notificar vía SNS (Riesgo de almacenamiento).
Red (VPC/ALB)	TargetResponseTime	> 2 segundos	Alerta de latencia en la aplicación.

3.3. Configuración de Alarmas y Notificaciones

Para cumplir con la métrica de **notificación automática**, se implementó el siguiente flujo:

1. **Creación de Alarma:** Se configuró una alarma en CloudWatch llamada `Alarma_CPU_Critico_ACME`.
2. **Servicio de Mensajería (SNS):** Se creó un tópico en **Amazon SNS** denominado `Alertas_Infraestructura`.
3. **Suscripción:** Se vinculó el correo corporativo del administrador al tópico SNS.
4. **Acción de Alarma:** Cuando la métrica de CPU supera el 80% (simulado mediante la prueba de carga con `stress`), el estado de la alarma cambia a `ALARM` y dispara automáticamente un correo electrónico.

3.4. Correlación de Incidentes y Logs

- **Gestión de Errores (SQS/DLQ):** En caso de fallos en el procesamiento de mensajes, se utiliza una **Dead Letter Queue (DLQ)** para capturar mensajes corruptos, evitando que bloqueen la cola principal.
- **Protocolo de Respuesta:**
    - **Paso 1:** Recepción de notificación SNS en el correo del administrador.
    - **Paso 2:** Revisión de métricas detalladas en el Dashboard de CloudWatch.
    - **Paso 3:** Inspección de la instancia EC2 o base de datos según la métrica afectada

4. Guía de Buenas Prácticas

Esta guía establece los lineamientos de diseño aplicados en el proyecto **"Infraestructura Viva"** para garantizar que la solución de ACME sea segura, eficiente y fácil de administrar.

4.1. Seguridad y Control de Acceso

- **Aislamiento de Red (VPC):** Se aplicó la segmentación de red para separar las capas de la aplicación. Las bases de datos (RDS) se alojan en **subredes privadas** sin acceso directo desde Internet.
- **Principio de Menor Privilegio (Security Groups):** Los Grupos de Seguridad actúan como firewalls virtuales. La base de datos solo permite tráfico entrante en el puerto 3306 desde el Grupo de Seguridad de la instancia EC2, restringiendo cualquier otra conexión no autorizada.
- **Protección de Contenido (OAC):** Para el alojamiento web, se configuró **Origin Access Control (OAC)** en CloudFront, asegurando que el bucket de S3 sea privado y que el contenido solo se entregue a través de la red de borde cifrada (HTTPS).

4.2. Escalabilidad y Rendimiento

- **Distribución de Carga:** Se implementó un **Application Load Balancer (ALB)** para gestionar el tráfico entrante de manera equilibrada, permitiendo que la infraestructura crezca horizontalmente según la demanda.
- **Bases de Datos de Alto Rendimiento:** * Uso de **Amazon RDS** con almacenamiento de estado sólido (SSD) para transacciones rápidas.
    - Uso de **Amazon DynamoDB** para datos semiestructurados, aprovechando su capacidad de escalar a millones de solicitudes por segundo con latencia de milisegundos.
- **Caché Global:** La implementación de **Amazon CloudFront** reduce la latencia al almacenar en caché los archivos estáticos en ubicaciones cercanas a los usuarios finales de ACME.

4.3. Resiliencia y Administración de Red

- **Alta Disponibilidad (Multi-AZ):** La base de datos relacional se configuró en modo **Multi-AZ**, lo que garantiza que, ante un fallo en una zona de disponibilidad, el sistema realice una conmutación por error automática a una réplica secundaria.
- **Desacoplamiento con Mensajería:** Mediante **Amazon SQS**, se logró separar la recepción de pedidos del procesamiento de los mismos. Esto evita que picos de tráfico saturen los servidores, ya que los mensajes se procesan al ritmo que la infraestructura permite.
- **Estrategia de Backup y Retención:** Se configuraron políticas de **versionamiento en S3** y ventanas de mantenimiento para backups automáticos en RDS, asegurando un **Punto de Recuperación (RPO)** mínimo.